UNIVERSITÉ DE MOUNDOU — TCHAD

Master en Informatique — Data Science et Intelligence Artificielle

RAPPORT DE PROJET

Gestion des inscriptions d'étudiants

Application CRUD en PHP et MySQL

Réalisé par : Mahamat Issa et Ibrahim Adam Fadoul

PHP · MySQL · phpMyAdmin · serveur local XAMPP

13 octobre 2026

Sommaire

1. Introduction et objectifs
2. Analyse des besoins et données
3. Architecture et environnement
4. Fonctionnalités réalisées
5. Validation et scénarios de test
6. Analyse critique et limites
7. Perspectives
8. Conclusion

1. Introduction et objectifs

Ce rapport présente une application web de gestion des étudiants, destinée à enregistrer et à suivre les inscriptions au sein d'une université. Elle suit le principe CRUD (Create, Read, Update, Delete) : créer un étudiant, consulter la liste, rechercher, modifier et supprimer une fiche.

Les objectifs de l'application sont :

- permettre la saisie des informations d'identité, de contact, d'origine géographique et de scolarité d'un étudiant ;
- afficher la liste des étudiants inscrits et retrouver rapidement une fiche grâce à une recherche par nom ;
- permettre la correction et la suppression d'une fiche ;
- informer l'utilisateur du résultat de chaque opération par un message de confirmation.

Le rapport décrit les besoins, la base de données, l'architecture, les écrans réalisés (captures à l'appui), les scénarios de validation, puis une analyse critique des limites et des améliorations possibles.

2. Analyse des besoins et données

2.1 Besoins fonctionnels

Fonction	Description	Écran
Créer	Enregistrer un nouvel étudiant à l'aide d'un formulaire.	Formulaire « Nouvel étudiant »
Lister	Afficher les étudiants avec leurs principales informations.	Page « Étudiants »
Rechercher	Retrouver un étudiant par son nom.	Champ « Rechercher un nom… »
Modifier	Corriger les informations d'une fiche.	Bouton « Modifier » de chaque ligne
Supprimer	Retirer une fiche de la base.	Bouton « Supprimer » de chaque ligne
Informer	Confirmer la réussite d'une opération.	Bandeau « Opération réussie. »

2.2 Données gérées

Les informations sont stockées dans la table etudiants de la base universite. Elle comporte douze colonnes, qui regroupent quatre familles d'informations :

Famille	Colonne	Signification
Identité	idEtu	Identifiant de l'étudiant, saisi comme « Matricule » dans le formulaire.
	nomEtu, prenomEtu	Nom et prénom.
	dateNaiss	Date de naissance.
Contact	emailEtu	Adresse email.
	numTelEtu	Numéro de téléphone.
Origine géographique	regEtu	Région.
	departEtu	Département.
	arrondisEtu	Arrondissement.
Scolarité	mentionEtu	Mention.
	cycleEtu	Cycle d'études (par exemple licence).
	ParcoursEtu	Parcours suivi (par exemple informatique).

L'origine géographique suit une hiérarchie région → département → arrondissement. Dans la version présentée, ces trois niveaux sont saisis librement dans des champs de texte.

3. Architecture et environnement

L'application suit une architecture client-serveur classique à trois niveaux : le navigateur affiche les pages et envoie les formulaires, le serveur web (Apache avec PHP) traite les requêtes, et MySQL stocke les données. L'ensemble s'exécute sur un poste local (adresse localhost), avec un serveur de type XAMPP, et la base est administrée avec phpMyAdmin 5.0.2.

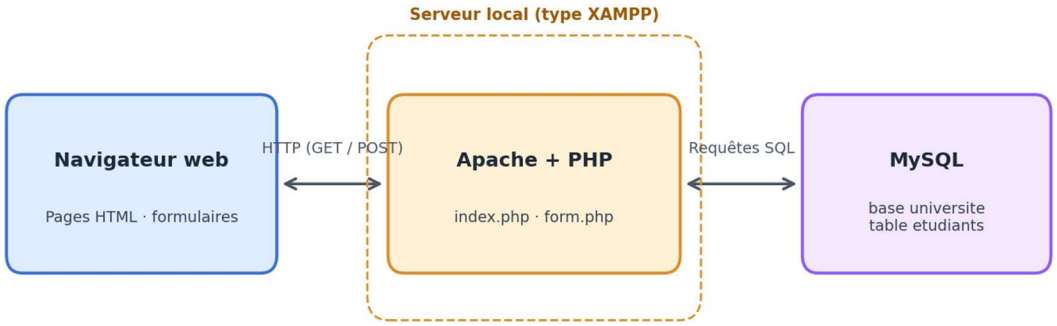


Figure 1 — Architecture de l'application

Élément	Description
Dossier de l'application	crud-minAppliInscriptionEtu, accessible à l'adresse http://localhost/crud-minAppliInscriptionEtu/.
index.php	Page « Gestion des étudiants » : liste, recherche, boutons d'action et messages de confirmation.
form.php	Page de saisie d'un étudiant (« Créer un étudiant » / « Nouvel étudiant »).
Base de données	MySQL, base universite, table etudiants (administration par phpMyAdmin).
Interface	Mise en page sobre de type Bootstrap : formulaire en carte, boutons colorés, tableau lisible.

Le paramètre ok=1 visible dans l'adresse de la page de liste (index.php?ok=1) montre que l'application revient à la liste après une opération et affiche alors un message de confirmation.

4. Fonctionnalités réalisées

4.1 Liste des étudiants et recherche

La page « Gestion des étudiants » présente un titre, un bouton « + Nouvel étudiant » en haut à droite, un champ de recherche par nom avec son bouton « Rechercher », puis le tableau des étudiants. Le tableau affiche le matricule, le nom, le prénom, l'email, la mention et le cycle, et chaque ligne dispose des boutons « Modifier » (bleu) et « Supprimer » (rouge). Après une opération, un bandeau vert affiche « Opération réussie. ».

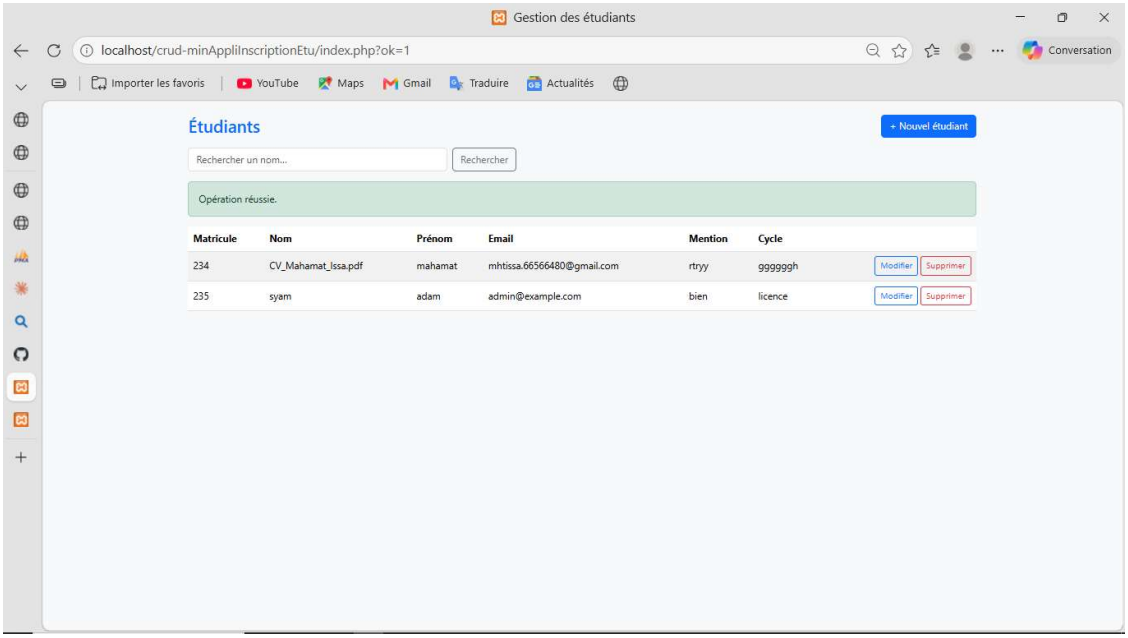


Figure 2 — Liste des étudiants, recherche et message de confirmation

4.2 Création d'un étudiant

Le formulaire « Nouvel étudiant » présente les champs les uns sous les autres : Matricule, Nom, Prénom, Date de naissance (avec sélecteur de date au format jj/mm/aaaa), Email, Téléphone, Région, Département, Arrondissement, puis les champs de scolarité plus bas dans la page (mention, cycle et parcours correspondent aux colonnes de la base). La mise en page en une colonne est lisible et adaptée à la saisie.

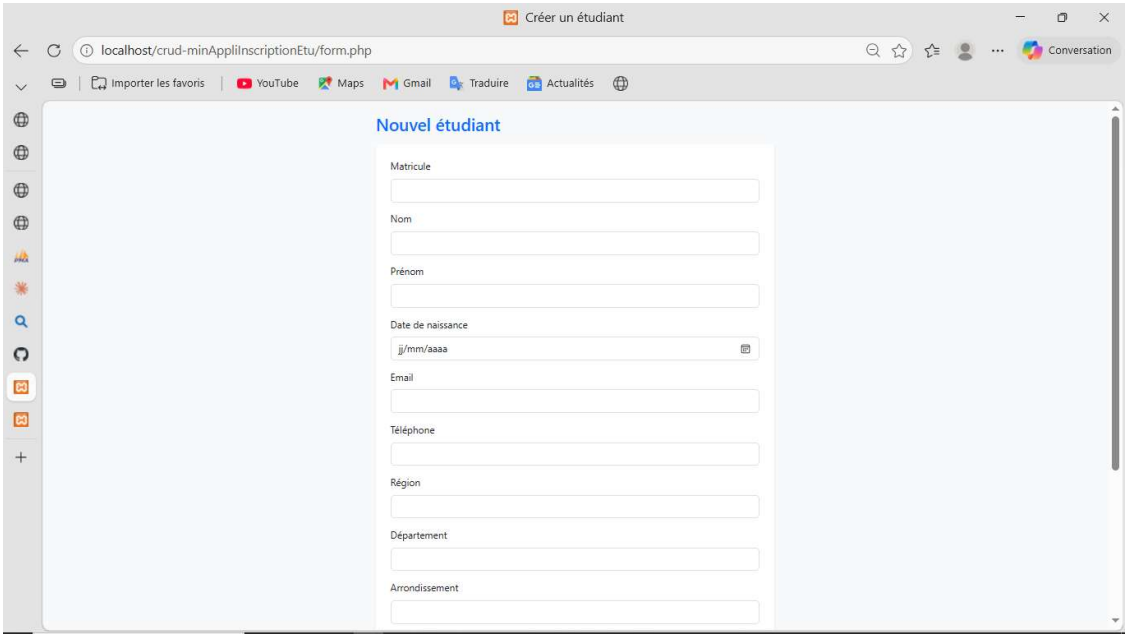


Figure 3 — Formulaire de création d'un étudiant

4.3 Modification et suppression

Chaque ligne de la liste propose deux actions : « Modifier », pour corriger les informations de la fiche, et « Supprimer », pour la retirer de la base. Ces actions sont directement accessibles depuis la liste, sans passer par une page intermédiaire de recherche.

4.4 Base de données

La capture phpMyAdmin montre le contenu de la table etudiants : deux enregistrements (idEtu 234 et 235) avec leurs douze colonnes. Ces deux lignes correspondent exactement à celles affichées dans la liste de l'application (matricule, nom, prénom, email, mention et cycle), ce qui confirme que l'interface lit bien les données de la base.

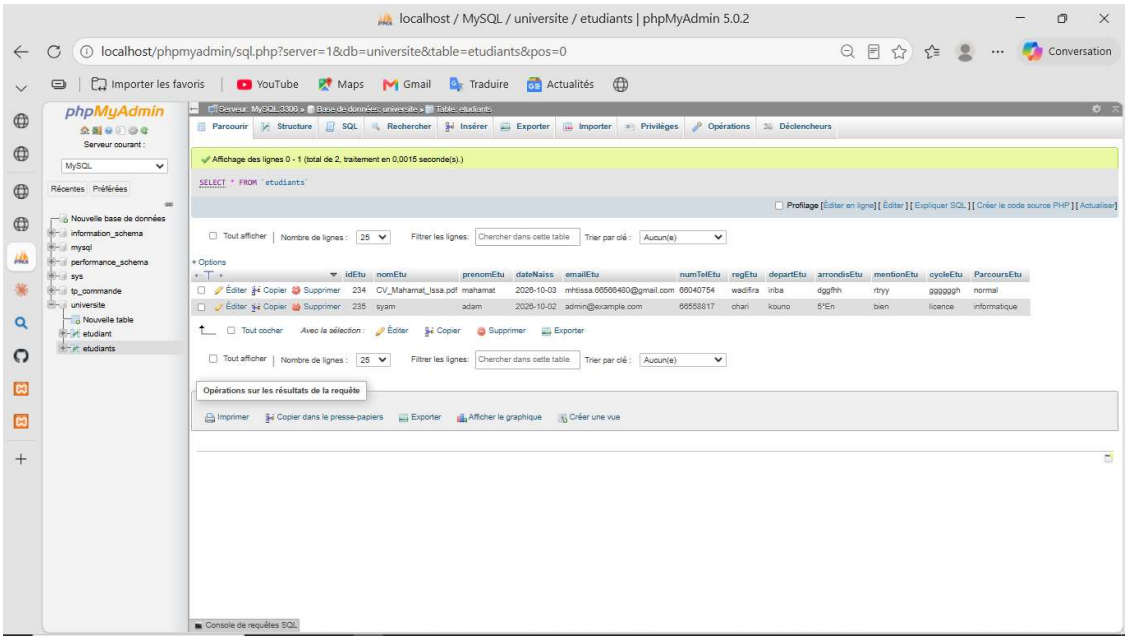


Figure 4 — phpMyAdmin — table etudiants de la base universite

La base universite contient également une table nommée etudiant (au singulier), qui n'apparaît pas dans les écrans présentés. Il sera préférable de n'en conserver qu'une seule pour éviter toute ambiguïté.

5. Validation et scénarios de test

Le tableau suivant distingue ce qui est établi par les captures d'écran de ce rapport et ce qui est présent à l'écran sans avoir été illustré.

Scénario	Résultat	Statut	Preuve
Affichage du formulaire de création	Les champs d'identité, de contact, de localisation et la date sont présents.	Observé	Figure 3
Enregistrement d'étudiants	Deux étudiants (matricules 234 et 235) sont présents dans la base et dans la liste.	Observé	Figures 2 et 4
Message de confirmation	Le bandeau « Opération réussie. » s'affiche après l'opération (adresse index.php?ok=1).	Observé	Figure 2
Cohérence liste / base	Les valeurs de la liste sont identiques à celles de la table etudiants.	Observé	Figures 2 et 4
Recherche par nom	Un champ de recherche et un bouton « Rechercher » sont disponibles.	Présent, non illustré	Figure 2
Modification d'une fiche	Un bouton « Modifier » est disponible sur chaque ligne.	Présent, non illustré	Figure 2
Suppression d'une fiche	Un bouton « Supprimer » est disponible sur chaque ligne.	Présent, non illustré	Figure 2

6. Analyse critique et limites

L'application remplit sa fonction de base, mais les données de test et les écrans montrent plusieurs points à renforcer avant un usage réel :

- **Contrôle des saisies :** la base contient une date de naissance située dans le futur (2026-10-03) et un nom contenant un nom de fichier (« CV_… .pdf »). Des contrôles de format et de cohérence (date plausible, caractères autorisés, email valide, téléphone) doivent être ajoutés, côté navigateur et surtout côté serveur.
- **Champs à valeurs libres :** les champs mention, cycle, parcours, région, département et arrondissement acceptent du texte libre (par exemple « rtryy » ou « ggggggh »). Des listes de valeurs fermées, avec des listes déroulantes dépendantes (région → département → arrondissement), garantiraient des données propres et exploitables pour des statistiques.
- **Unicité :** le matricule et l'adresse email devraient être uniques (clé primaire sur le matricule, contrainte UNIQUE sur l'email) pour éviter les doublons.
- **Sécurité :** il convient de vérifier l'usage de requêtes préparées (protection contre l'injection SQL), de l'échappement des données affichées (protection contre les failles XSS) et d'un jeton de protection des formulaires. L'accès à l'application devrait aussi être réservé à des utilisateurs authentifiés.
- **Suppression :** une demande de confirmation avant suppression, ou une suppression « logique » (étudiant marqué comme archivé), éviterait les pertes de données par erreur.
- **Liste :** la pagination et le tri par colonne deviendront nécessaires avec un grand nombre d'étudiants.
- **Données personnelles :** les informations de contact (email, téléphone) sont des données personnelles : l'accès doit être restreint et les captures ou jeux d'essai ne doivent pas contenir de vraies coordonnées.

7. Perspectives

- authentification des utilisateurs et gestion de rôles (administrateur, scolarité, consultation) ;
- listes déroulantes dépendantes pour la localisation et listes fermées pour la scolarité ;
- ajout de la photo de l'étudiant et d'un dossier de pièces justificatives, avec contrôle du type et de la taille des fichiers ;
- pagination, tri, filtres combinés (cycle, parcours, région) et export de la liste en PDF ou Excel ;
- tableau de bord statistique : effectifs par cycle, par parcours, par région d'origine ;
- migration vers un framework (par exemple Laravel) pour bénéficier de la validation, de la sécurité et de la structuration du code intégrées.

8. Conclusion

L'application de gestion des étudiants met en œuvre les opérations essentielles d'un CRUD sur une base MySQL : saisie d'une fiche étudiante, liste, recherche par nom, modification, suppression et message de confirmation. Les captures montrent une interface claire et des données cohérentes entre l'application et la base.

Les principales améliorations à apporter concernent la qualité des données (validation, listes fermées, unicité), la sécurité (authentification, requêtes préparées, protection des formulaires) et l'ergonomie (pagination, confirmation de suppression). Elles constituent la suite logique du projet.